Python Engineering - Class 04

Funciones y descomposicion | Guía de ejercicios

Objetivo de la clase: Transformar scripts monoliticos en funciones con entradas, salidas y responsabilidades claras.

Este documento es la guía base del ejercicio. No contiene una solución completa para copiar: organiza el problema, los pasos de implementación, los valores de referencia y la validación.

Ruta

```
01_python_bootcamp/
`-- week_01_logic/class_04_functions/
    |-- notes/
    `-- exercises/
```

Flujo de trabajo obligatorio

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Antes de programar, redacta estas secciones. El objetivo es separar el razonamiento de la sintaxis.

EX09 - Presion dinamica como funcion

Problema: Encapsular la ecuacion de presion dinamica en una funcion reutilizable.

Entradas de referencia: rho, speed

Ecuaciones / relación: q=0.5*rho*speed**2

Comprobación esperada: rho=1.225 y speed=100 -> 6125 Pa.

Guía de implementación

- 1. Definir parametros.
- 2. Calcular q dentro de la funcion.
- 3. Return q.
- 4. Llamar la funcion con distintos valores.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

EX10 - Ecuacion del cohete como funcion

Problema: Separar validacion y calculo de Delta_v en funciones.

Entradas de referencia: Isp, g0, m0, mf

Ecuaciones / relación: Delta_v=Isp*g0*ln(m0/mf)

Comprobación esperada: Utilizar una referencia válida de EX02 y comprobar que la función devuelve el mismo resultado.

Guía de implementación

- 1. Crear validate_inputs.
- 2. Usar return booleano.

- 3. Calcular solo si la validacion pasa.
- 4. Devolver Delta_v.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

EX11 - Taylor exp como funcion

Problema: Convertir el algoritmo iterativo de Taylor en una funcion parametrica.

Entradas de referencia: x, N

Ecuaciones / relación: sum(x**n/n!)

Comprobación esperada: Comparar con math.exp para varios valores de N.

Guía de implementación

- 1. Recibir x y N.
- 2. Construir acumulacion local.
- 3. Return aproximacion.
- 4. Comparar varias llamadas.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

EX12 - Movimiento vertical con funciones

Problema: Separar validacion, paso Euler, simulacion y solucion analitica.

Entradas de referencia: h0, v0, t0, g, dt

Ecuaciones / relación: Euler + fórmulas balísticas analíticas

Comprobación esperada: Las funciones deben devolver datos y no depender innecesariamente de globales. La solución analítica no depende de dt.

Guía de implementación

- 1. Crear validate_inputs().
- 2. Crear euler_step().
- 3. Crear simulacion.
- 4. Crear solucion analitica.
- 5. Comparar resultados.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

Validación final

Responde estas preguntas como comentarios al final del ejercicio. El objetivo es demostrar que puedes explicar el flujo del código.

- Diferencia entre parametro y argumento.
- Diferencia entre print y return.
- Que es scope local?
- Por que euler_step devuelve tres valores?
- Por que la solucion analitica no debe depender de dt?
- Que ventaja tiene separar validate_inputs de la funcion fisica?

Criterio de cierre

La clase se considera completada cuando el código se ejecuta, las comprobaciones de referencia tienen sentido, puedes explicar cada bloque sin leer una solución y puedes modificar al menos un input anticipando su efecto.